AI CODE SECURITY ANALYZER · TEAM 7 · PHASE 2

Architecture pipeline, input to output

AI-generated code goes in. A verdict, the evidence and a fix list come out. Two analyses run on the same code: Joern follows user input through the program, and syntax-tree checks look for AI-specific mistakes.

INPUT

AI-generated code
A pasted snippet with the prompt that produced it, or a zipped Python project.

FRONTEND

Web page — `React + Vite`
Collects the code and sends it to the backend.
`frontend/src/components/Evaluator.jsx`

HTTP: JSON to /analyze, or zip to /analyze-project

BACKEND

API and validation — `FastAPI + Pydantic`
Rejects empty, oversized or unsupported input, unpacks zips safely, then parses each file into a syntax tree. Code that does not parse is rejected here.
`backend/main.py`

same code to both analyses

ANALYSIS

Security analysis — `Joern in Docker`
Follows data through functions and across files.

- **Build the Code Property Graph**
  Structure, control flow and data flow in one graph.
- **Mark sources and sinks**
  Sources: web requests, `input()`, command-line args. Sinks: SQL, shell, `eval`, HTML responses.
- **Taint query**
  Find every path from a source to a sink.

Finds: SQL, command and code injection, and reflected XSS, each with its path.
`joern_engine.py · taint_queries/python_taint.sc`

If Joern is not running, a simpler single-file scan runs instead and the report says so. `ast_fallback.py`

AI-mistake checks — `Python AST`
Run directly on the syntax tree.

- **Hallucinated APIs**
  Calls to functions that do not exist, checked against the real libraries.
- **Logic errors**
  Off-by-one, missing return, infinite loop and similar.
- **Context gaps**
  Names or project files the code assumes but was never given.

`checks.py`

findings from all four checks

DECISION

Verdict and recommendation — `Python`
Combines the findings into one verdict and an ordered fix list. Names, CWE ids and fix text come from the rules table.

Reject · security issue or made-up API | **Review** · logic error or context gap | **Accept** · nothing found

`analyzer.py · rules.py`

JSON report

OUTPUT

Report in the web page

- Verdict and reason
- "What to do": fixes in priority order, with file and line
- Findings by category, with CWE number and severity
- Taint path for each vulnerability, from source to sink

`frontend/src/components/Report.jsx`

Runs locally
The page, the backend and Joern all run on one machine. Submitted code does not leave it.

How the backend reaches Joern
HTTP requests to Joern's server on port 8080, with code and results passed through the shared joern_workspace folder.

Current scope
Python is fully analyzed. JavaScript gets a basic pattern scan and skips Joern.